BENCHMARK REPRODUCIBLE

Multiplicación de Matrices NxN

C++ | Go | Java | Python

Experimento Computacional Reproducible
ACM Artifact Review & Badging

Octubre 2026

01

Introducción

¿Qué es este proyecto y por qué es importante?

¿Qué hicimos?



4 Lenguajes

C++, Go, Java, Python



3 Tamaños

N = 500, 1000, 1500



10 Repeticiones

Por combinación



120 Mediciones

Total de observaciones

Pregunta de investigación

¿Existen diferencias estadísticamente significativas en el tiempo de ejecución del algoritmo naive de multiplicación de matrices NxN entre implementaciones equivalentes en C++, Go, Java y Python?

H_0: No hay diferencias significativas entre lenguajes

H_1: Al menos un lenguaje es significativamente diferente

02

Metodología

¿Cómo se diseñó y ejecutó el experimento?

Diseño experimental

1

Algoritmo

Multiplicación naive $O(n^3)$
Triple bucle anidado idéntico en los 4 lenguajes

2

Datos de entrada

Generador LCG con semilla fija (seed=42 y seed=43)
Matrices idénticas en todos

3

Medición

Solo se cronometra la multiplicación. Generación e inicialización excluidas

Herramientas de reproducibilidad

Docker

Contenedores aislados por lenguaje
gcc:13 | golang:1.22
eclipse-temurin:21 | python:3.12

Snakemake

Pipeline de automatización
Ejecuta los 120 benchmarks
y el análisis estadístico

Git + GitHub

Control de versiones
Repositorio público
github.com/jffp-alt/matrix-benchmark

03

Resultados

¿Qué encontramos?

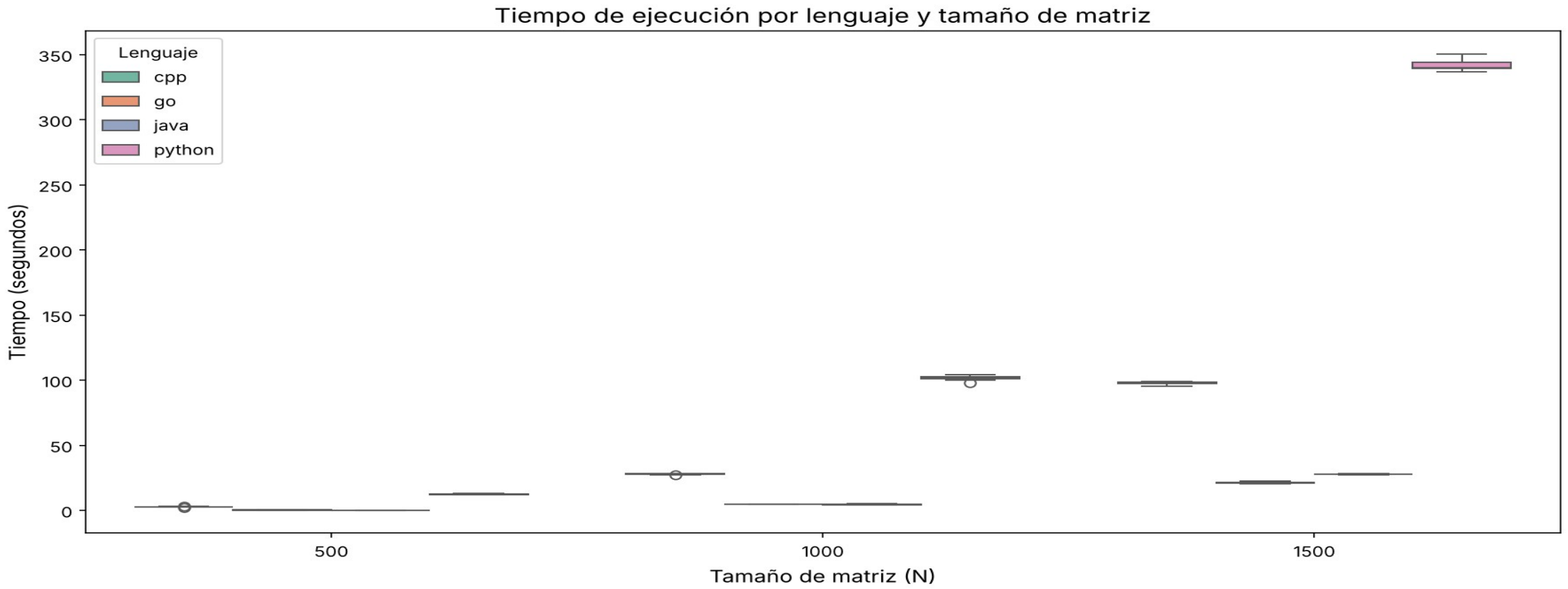
Tiempos promedio de ejecución (segundos)

Lenguaje	N=500	N=1000	N=1500
C++	2.73	28.00	97.83
Go	0.32	4.72	21.42
Java	0.26	4.65	27.80
Python	12.46	101.88	**342.08**

Más rápidos
Go y Java: ~0.3s para N=500
Prácticamente equivalentes

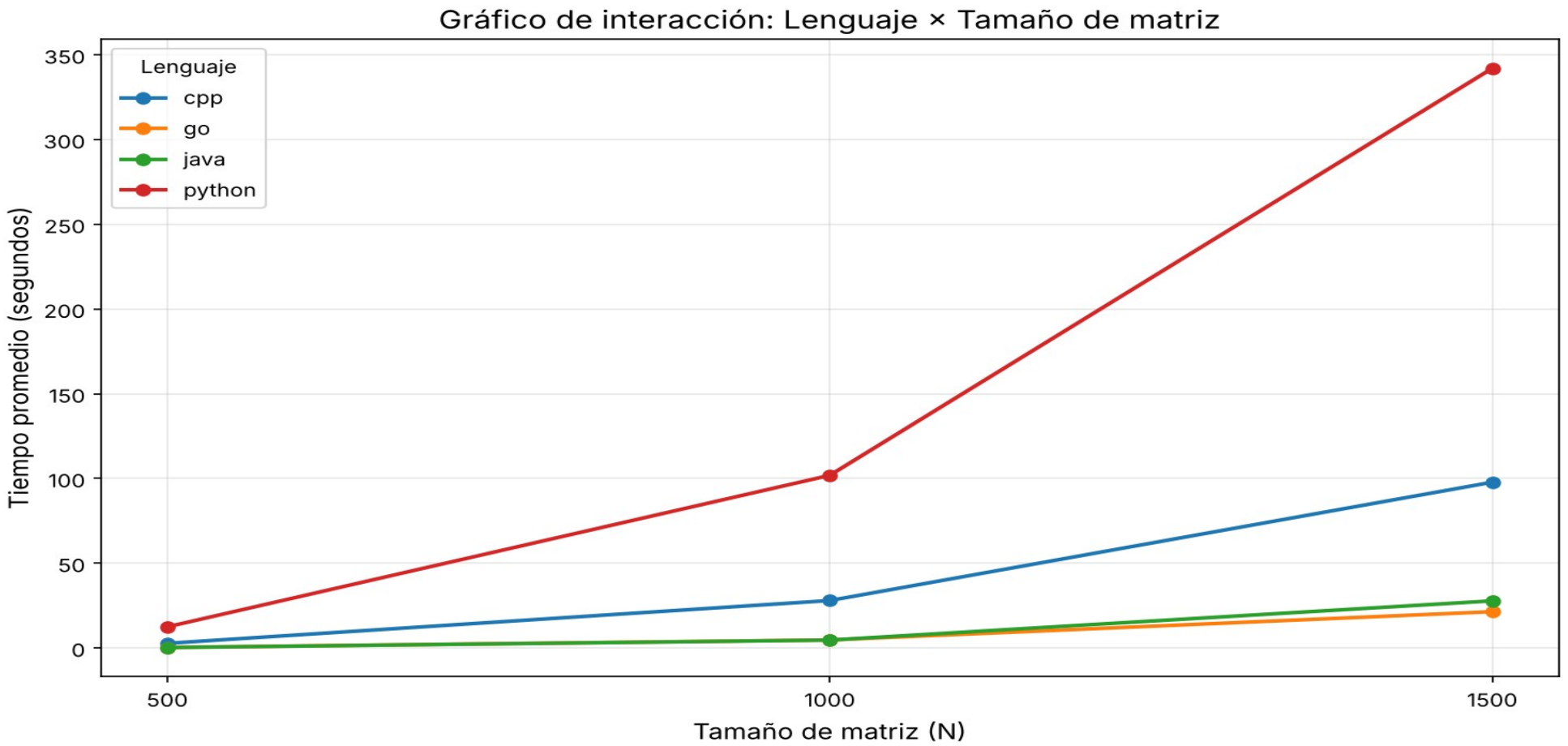
Más lento
Python: 342s para N=1500
16x más lento que Go

Distribución de tiempos (Boxplot)



Python domina los tiempos altos. Go y Java son los más rápidos con baja variabilidad.

Gráfico de interacción: Lenguaje × Tamaño



Las líneas no son paralelas → interacción significativa. Python crece mucho más rápido con N.

04

Análisis estadístico

ANOVA + Tukey HSD

ANOVA de dos factores con interacción

Factor	F	p-valor	Resultado
Lenguaje	62,317	1.09e-174	Significativo ***
Tamaño (N)	68,794	2.01e-168	Significativo ***
Lenguaje × N	25,490	1.03e-167	Significativo ***

Todos los factores son altamente significativos **(p < 0.001)**

El lenguaje, el tamaño de matriz y su interacción afectan significativamente el tiempo de ejecución.

Comparaciones Tukey HSD

Python vs Go	Δ 143.3s	p = < 0.001	**Significativa**
Python vs Java	Δ 141.2s	p = < 0.001	**Significativa**
Python vs C++	Δ 109.3s	p = < 0.001	**Significativa**
C++ vs Go	Δ 34.0s	p = 0.289	**No significativa**
C++ vs Java	Δ 31.9s	p = 0.344	**No significativa**
Go vs Java	Δ 2.1s	p = 0.999	**No significativa**

05

Conclusiones

Hallazgos principales

Se rechaza H_0: Sí existen diferencias significativas entre lenguajes

1. Go y Java son los más rápidos, sin diferencias significativas entre ellos
2. C++ sin optimizaciones (-O0) tiene rendimiento intermedio
3. Python (CPython) es ~16x más lento que Go/Java para N=1500
4. La brecha de rendimiento se amplifica con matrices más grandes

Criterios ACM Artifact

Functional

El código se ejecuta
y produce los resultados
documentados

Reusable

Docker + Snakemake
Semilla fija
Documentación completa

Available

GitHub público
Licencia MIT
Zenodo (pendiente DOI)

github.com/jffp-alt/matrix-benchmark